CS24301 — THEORY (3.0 CR) FULL COURSE MASTER BIT MESRA | NEP SCHEME

Compiler Design (CS24301) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

Module I: Lexical Analysis — Topics Covered

1. Language Processing Systems & Cousins of Compiler
2. 6 Phases of Compiler with Running Example
3. Token, Pattern, Lexeme & Token Attributes
4. Input Buffering (Buffer Pairs & Sentinels)
5. Thompson's Construction (RE → NFA)
6. Subset Construction (NFA → DFA)
7. Hopcroft's DFA State Minimization
8. Direct DFA Construction (Syntax Tree Method)

1. Language Processors & 6 Phases of Compiler

A **compiler** translates high-level source code into target machine language while reporting errors.

Tool	Input → Output	Core Responsibilities
Preprocessor	`.c` → `.i` (Pure Source)	Macro expansion (`#define`), header inclusion (`#include`), stripping comments.
Compiler	`.i` → `.s` (Assembly)	Front-end analysis & back-end code synthesis.
Assembler	`.s` → `.o` (Relocatable Object)	Translates mnemonics to machine code and symbol offsets.
Linker	`.o` + Libs → `.exe`	Resolves external symbol references across modules.
Loader	Binary → RAM	Allocates memory segments, sets up stack/heap, jumps to `main` .

2. Input Buffering & Sentinels

Using **Buffer Pairs** ($2 \times N$ characters) with **Sentinel (`EOF`)** characters placed at buffer ends eliminates one conditional check per character, reducing inner scanning loop overhead by 50%.

3. Direct DFA Construction (Syntax Tree Method)

Constructs DFA directly from augmented regular expression $(r)\#$:

- `nullable(n)` : True iff subtree generates ϵ.
- `firstpos(n)` : Set of positions matching the first character of subtree strings.
- `lastpos(n)` : Set of positions matching the last character of subtree strings.
- `followpos(i)` : Positions immediately following position i.
  - For cat-node $c_1 \cdot c_2$: add $\text{firstpos}(c_2)$ to $\text{followpos}(i)$ for all $i \in \text{lastpos}(c_1)$.
  - For star-node c_1^*: add $\text{firstpos}(c_1)$ to $\text{followpos}(i)$ for all $i \in \text{lastpos}(c_1)$.

BIT Mesra Exam Solved Problem: $(a|b)^*abb\#$

Leaves: $a(1), b(2), a(3), b(4), b(5), \#(6)$.

$\text{followpos}(1) = \{1,2,3\}, \text{followpos}(2) = \{1,2,3\}, \text{followpos}(3) = \{4\}, \text{followpos}(4) = \{5\}, \text{followpos}(5) = \{6\}, \text{followpos}(6) = \emptyset$.

DFA States: $A = \{1,2,3\} \xrightarrow{a} B = \{1,2,3,4\} \xrightarrow{b} C = \{1,2,3,5\} \xrightarrow{b} D^* = \{1,2,3,6\}$.

Module II: Syntax Analysis — Topics Covered

1. Role of Parser & Context-Free Grammars (CFG)
2. Ambiguity & Derivations (LM / RM)
3. Grammar Rewriting (Left Recursion & Factoring)
4. Top-Down Parsing: LL(1) Table Construction
5. Computation of FIRST and FOLLOW Sets
6. Bottom-Up Parsing & Handle Pruning
7. LR Parsers: LR(0), SLR(1), CLR(1), LALR(1)
8. Parsing Conflicts & Error Recovery Strategies

1. Context-Free Grammars (CFG) & Ambiguity

A **Context-Free Grammar** is a 4-tuple $G = (V, \Sigma, R, S)$ where V is a finite set of non-terminal variables, Σ is a finite set of terminal symbols ($V \cap \Sigma = \emptyset$), R is a finite set of production rules of the form $A \to \alpha$ ($A \in V, \alpha \in (V \cup \Sigma)^*$), and $S \in V$ is the start symbol.

Leftmost vs. Rightmost Derivations

Leftmost Derivation (LMD): At each step, the leftmost non-terminal is replaced first.
Rightmost Derivation (RMD / Canonical Derivation): At each step, the rightmost non-terminal is replaced first.
Ambiguous Grammar: A grammar that produces *two or more distinct parse trees* (or distinct LMDs) for at least one string $w \in L(G)$. (Classic example: "Dangling-Else" ambiguity).

2. Grammar Transformations for Top-Down Parsing

2.1 Elimination of Immediate Left Recursion

A production rule $A \to A\alpha \mid \beta$ is immediately left-recursive. It causes top-down predictive parsers to enter an infinite loop. We eliminate it by introducing a new non-terminal A':

$$A \to \beta A', \quad A' \to \alpha A' \mid \epsilon$$

2.2 Left Factoring (Resolving Common Prefixes)

When two productions for A share a common leading prefix $A \to \alpha\beta_1 \mid \alpha\beta_2$, a predictive parser cannot decide which production to choose based on 1 lookahead token. We left-factor:

$$A \to \alpha A', \quad A' \to \beta_1 \mid \beta_2$$

3. Computation of FIRST and FOLLOW Sets

Set	Mathematical Definition	Construction Rules
$\mathrm{FIRST}(\alpha)$	$\mathrm{FIRST}(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta\}$ (includes ϵ if $\alpha \Rightarrow^* \epsilon$)	1. If $X \in \Sigma, \mathrm{FIRST}(X) = \{X\}$. 2. If $X \to \epsilon$, add $\epsilon \in \mathrm{FIRST}(X)$. 3. For $X \to Y_1 Y_2 \dots Y_k$: add $\mathrm{FIRST}(Y_1) \setminus \{\epsilon\}$. If $Y_1 \Rightarrow^* \epsilon$, add $\mathrm{FIRST}(Y_2) \setminus \{\epsilon\}$, continuing until a non-nullable Y_i is reached. If all $Y_i \Rightarrow^* \epsilon$, add ϵ.
$\mathrm{FOLLOW}(A)$	$\mathrm{FOLLOW}(A) = \{a \in \Sigma \cup \{\$\} \mid S \Rightarrow^* \alpha A a \beta\}$	1. Add $\$$ to $\mathrm{FOLLOW}(S)$ (where S is start symbol). 2. If $A \to \alpha B \beta$, add $\mathrm{FIRST}(\beta) \setminus \{\epsilon\}$ to $\mathrm{FOLLOW}(B)$. 3. If $A \to \alpha B$ or $A \to \alpha B \beta$ where $\epsilon \in \mathrm{FIRST}(\beta)$, add all elements of $\mathrm{FOLLOW}(A)$ to $\mathrm{FOLLOW}(B)$.

4. Top-Down $\mathrm{LL}(1)$ Parsing Table Construction

For each production $A \to \alpha$:

1. For every terminal $a \in \text{FIRST}(\alpha)$, add $A \to \alpha$ to $M[A, a]$.
2. If $\epsilon \in \text{FIRST}(\alpha)$, then for every terminal $b \in \text{FOLLOW}(A)$ (including \$), add $A \to \alpha$ to $M[A, b]$.

LL(1) Condition: A grammar is $\text{LL}(1)$ iff no cell in parsing table M contains multiple entries (no conflicts).

5. Bottom-Up LR Parsers ($\text{LR}(0), \text{SLR}(1), \text{CLR}(1), \text{LALR}(1)$)

Parser Type	Item Representation	Lookahead & Table Size	Power & Conflict Resolution
$\text{LR}(0)$	$A \to \alpha \cdot \beta$	0 lookaheads. Same number of states as SLR(1).	Weakest; frequent Shift/Reduce conflicts on reduction.
$\text{SLR}(1)$	$A \to \alpha \cdot \beta$	Place reduction $A \to \alpha$ only in $\text{FOLLOW}(A)$.	More powerful than $\text{LR}(0)$, but still suffers conflicts on ambiguous grammars.
$\text{CLR}(1)$	$[A \to \alpha \cdot \beta, a]$ where $a \in \Sigma \cup \{\$\}$	Carries exact lookahead a. Huge state table.	Most powerful LR parser for deterministic CFGs. Large memory footprint.
$\text{LALR}(1)$	Merged $\text{CLR}(1)$ items with identical core	Same number of states as $\text{SLR}(1)$, lookaheads merged.	**Standard in Yacc/Bison**; rarely introduces R/R conflicts, never introduces S/R conflicts.

🏛 **BIT Mesra Exam Question & Solved Walkthrough (10 Marks)**

Question: Construct the $\text{LL}(1)$ parsing table for the grammar: $E \to TE', E' \to +TE' \mid \epsilon, T \to FT', T' \to *FT' \mid \epsilon, F \to (E) \mid \text{id}$.

Solution:

- $\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{(, \text{id}\}$
- $\text{FIRST}(E') = \{+, \epsilon\}, \quad \text{FIRST}(T') = \{*, \epsilon\}$
- $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$\}$
- $\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{+,), \$\}$
- $\text{FOLLOW}(F) = \{*, +,), \$\}$

Every cell in table M contains at most one production $\implies$ The grammar is strictly $\text{LL}(1)$.

Module III: Semantic Analysis & Intermediate Code Generation — Topics Covered

1. Syntax-Directed Definitions (SDD) & Attributes
2. S-Attributed vs. L-Attributed Definitions
3. Syntax-Directed Translation Schemes (SDTS)
4. Type Systems, Checking & Coercion
5. Three-Address Code (TAC): Quadruples & Triples
6. Multi-Dimensional Array Address Calculations

1. Syntax-Directed Definitions (SDD)

An **SDD** is a Context-Free Grammar augmented with attributes and semantic rules.

Attribute Category	Definition & Dependency Order	Evaluation Paradigm
Synthesized Attribute	Computed solely from the values of attributes at the child nodes in the parse tree: $A.s = f(Y_1.a, Y_2.b, \ldots, Y_k.c)$.	Evaluated naturally during **Bottom-Up Parsing** (Post-order traversal).
Inherited Attribute	Computed from attributes of the parent node and/or sibling nodes: $Y_j.i = f(A.a, Y_1.b, \ldots, Y_{j-1}.c)$.	Evaluated during **Top-Down Parsing** (Pre-order / In-order traversal).

S-Attributed vs. L-Attributed SDD Classification

S-Attributed SDD: Uses *exclusively synthesized attributes*. Can be evaluated directly in bottom-up LR parsers without building explicit trees.
L-Attributed SDD: Allows synthesized attributes AND inherited attributes, provided inherited attributes of Y_j depend only on A's inherited attributes or attributes of siblings to the *left* ($Y_1, \ldots, Y_{j-1}$). Evaluated in a single Depth-First Search (DFS) left-to-right pass.

2. Three-Address Code (TAC) Representations

Three-Address Code instructions have at most one operator on the RHS: $x = y \text{ op } z$.

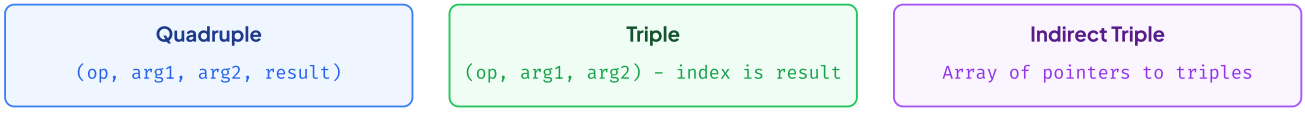

FIGURE 3.1: COMPARISON OF THREE-ADDRESS CODE DATA STRUCTURES

3. Multi-Dimensional Array Address Calculations

Consider an array $A[d_1][d_2]$ with element size w and base address Base. Let lower bounds be low_1, low_2:

Memory Storage Order	Address Formula for Element $A[i][j]$
Row-Major Order (C/C++, Java)	$\text{Address}(A[i][j]) = \text{Base} + \Big((i - low_1) \times n_2 + (j - low_2)\Big) \times w$
Column-Major Order (Fortran, MATLAB)	$\text{Address}(A[i][j]) = \text{Base} + \Big((j - low_2) \times n_1 + (i - low_1)\Big) \times w$

🏛️ BIT Mesra Numerical: 2D Array TAC Generation

Problem: Generate Three-Address Code for the assignment `x = A[i][j]` where A is a 10×20 integer array ($w = 4$ bytes, $low_1 = 1, low_2 = 1$) stored in Row-Major order.

Derivation:

```
// Address formula: Base + ((i - 1)*20 + (j - 1))*4
t1 = i - 1
t2 = t1 * 20
t3 = j - 1
t4 = t2 + t3
t5 = t4 * 4
t6 = A[t5]      // Array dereference with offset t5
x = t6
```

Module IV: Advanced ICG & Runtime Environment — Topics Covered

1. Short-Circuit vs. Numerical Boolean Evaluation
2. Control Flow Translation (if-else, while, for)
3. Backpatching: makelist, merge, backpatch
4. Procedure Calls & Return Mechanisms
5. Runtime Memory Organization (Text, Data, Heap, Stack)
6. Activation Records (Stack Frames) & Parameter Passing

1. Translation of Boolean Expressions (Short-Circuit Evaluation)

In programming languages like C and Java, boolean operators `&&` and `||` evaluate conditionally:

- In $B_1 \ || \ B_2$: If B_1 is true, B_2 is skipped entirely.
- In $B_1 \ \&\& \ B_2$: If B_1 is false, B_2 is skipped entirely.

2. Backpatching Technique

In a single-pass compiler, forward jump targets are not known when the jump instruction is emitted. **Backpatching** leaves target addresses blank and fills them once the target label is generated.

- `makelist(i)` : Creates a new list containing just the jump instruction index i.
- `merge(p1, p2)` : Concatenates two lists of jump instructions $p1$ and $p2$.
- `backpatch(p, i)` : Inserts label i as the target for each instruction on list p.

3. Runtime Storage Organization & Activation Records

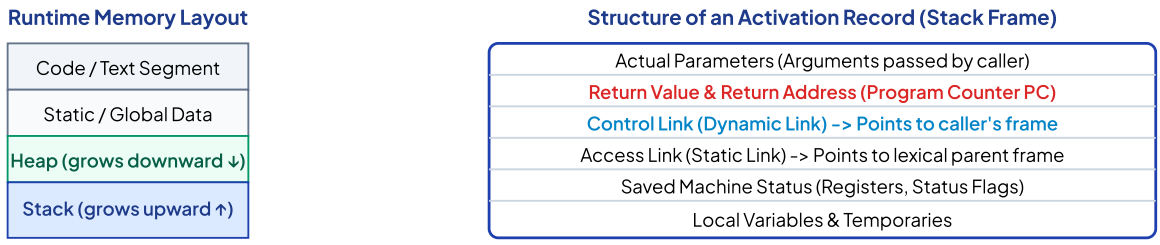

FIGURE 4.1: MEMORY LAYOUT & ACTIVATION RECORD ARCHITECTURE

4. Parameter Passing Mechanisms

Mechanism	How Arguments are Passed	Effect of Changes on Actual Arguments
Call by Value	Evaluates r-value and passes a copy to local parameter.	Changes to formal parameter have **zero effect** on caller.
Call by Reference	Passes l-value (memory address / pointer) of actual argument.	Changes directly **mutate caller's variable**.
Call by Name	Textual macro substitution (evaluated lazily via Thunk).	Re-evaluated each time parameter is referenced.

Module V: Code Optimization & Target Generation — Topics Covered

1. Basic Blocks & Leader Instruction Algorithm
2. Control Flow Graph (CFG) & Loop Dominators
3. DAG Construction & Local Optimizations
4. Machine-Independent Global Optimizations
5. Register Allocation (Graph Coloring Heuristics)
6. Peephole Optimization Techniques

1. Basic Blocks & Leader Identification Algorithm

A **Basic Block** is a sequence of consecutive Three-Address Code statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching except at the end.

Algorithm: Identifying Leader Instructions

1. The **first statement** in the TAC program is a leader.
2. Any statement that is the **target of a conditional or unconditional goto** is a leader.
3. Any statement that **immediately follows a conditional or unconditional goto** is a leader.

2. Directed Acyclic Graph (DAG) for Basic Blocks

A DAG represents basic block expressions without redundancy:

- Leaves represent initial variable values or constants.
- Interior nodes represent operators.
- Nodes are annotated with variable labels currently holding the computed value.

Optimization Technique	Description & Realized Benefits
Common Subexpression Elimination (CSE)	Identifies identical subexpression computations whose operand values have not changed, eliminating redundant operations.
Constant Folding	Evaluates operations with constant operands at *compile-time* (e.g., replacing `2 * 3.1415 * r` with `6.283 * r`).
Constant Propagation	Replaces variable uses with known compile-time constants.
Dead Code Elimination	Deletes instructions that compute values never used on any execution path.
Loop-Invariant Code Motion (Hoisting)	Moves computations whose operands are constant throughout the loop to the loop pre-header.
Induction Variable Elimination & Strength Reduction	Replaces expensive multiplications (e.g., $i \times 4$) inside loops with incremental additions ($t = t + 4$).

3. Register Allocation via Graph Coloring (Chaitin's Algorithm)

Given k hardware registers, we construct the **Interference Graph** where nodes are variables/temporaries and edges represent live ranges that overlap. If the graph can be colored with k colors such that no two adjacent nodes share the same color, no register spilling occurs.

4. Peephole Optimization

Examines a short sliding window (peephole) of target instructions to apply local transformations:

- **Redundant Load/Store Elimination:** `MOV R0, x` followed by `MOV x, R0` $\rightarrow$ remove 2nd move.

- **Unreachable Code Elimination:** Removing instructions immediately following unconditional jumps.
- **Flow-of-Control Optimizations:** `goto L1 ... L1: goto L2` → replace with `goto L2`.
- **Algebraic Simplifications:** Replacing `x = x + 0` or `x = x * 1` with no-op; replacing `x = x * 2` with `x = x << 1`.
